

Step 2 — A Real ReAct Agent That Calls a Discovered Tool — Hints

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints. Each has 3 depth levels: **Basic** (the plain idea), **Intermediate** (the real OpenAI + `mcp` calls), **Advanced** (the edge cases a discovered, not hand-written, tool set actually creates). Read Basic first even if Doc06/07 already feel familiar — the new part here is specifically the *bridge* between the two protocols, not the loop itself.

- [Hint 1 — Turning a discovered tool into something OpenAI can call](#)
- [Hint 2 — The loop itself, and almost the whole thing](#)

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — Turning a discovered tool into something OpenAI can call

Basic Version

OpenAI's function calling and MCP's tools are two different protocols describing almost the exact same information: a name, a description, and a shape the arguments have to match. Your job is to translate one shape into the other — not to write anything new about what the tool actually does, since you never wrote the tool at all.

Two small translation functions are all this needs: - One that turns *every discovered MCP tool* into *one OpenAI tool definition*, so the whole list can be handed to the model at once. - One that turns *a tool call the model asked for* into *an actual MCP tool call*, and turns whatever comes back into plain text the model can read.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Each MCP `Tool` object (from Step 1) has `.name`, `.description`, and `.inputSchema`. OpenAI's `tools` parameter wants a list of `{"type": "function", "function": {"name": ..., "description": ..., "parameters": ...}}` dicts, where `parameters` is JSON Schema. Since `.inputSchema` is *already* JSON Schema, the conversion is close to a direct pass-through:

```
def mcp_tools_to_openai_schema(tools):
    return [
        {
            "type": "function",
            "function": {
                "name": tool.name,
                "description": tool.description or "",
                "parameters": tool.inputSchema,
            },
        }
        for tool in tools
    ]
```

Calling a tool the model picked is the reverse direction. `await session.call_tool(name, arguments)` returns a `CallToolResult`. Its `.content` is a *list* of content blocks — usually `TextContent` objects with a `.text` attribute — because MCP tool results can in principle carry more than one block, or non-text content. For a plain string the model can read, join the text blocks together:

```
async def call_mcp_tool(session, name, arguments):
    result = await session.call_tool(name, arguments)
    parts = [block.text for block in result.content if hasattr(block, "text")]
    return " ".join(parts) if parts else "(tool returned no text content)"
```

The tool call the model asks for arrives as `message.tool_calls`, a list where each entry has `.id`, `.function.name`, and `.function.arguments` — that last one is a **JSON string**, not a dict, so it needs `json.loads(...)` before you can hand it to `call_mcp_tool`.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Two things only show up once your tools are *discovered* instead of hand-written, and both are worth thinking through now, before Hint 2's loop.

A tool's `.description` can be empty or missing. A hand-written `@tool` function always has a docstring you wrote yourself — you'd never ship one blank. A tool discovered from someone else's server might genuinely have `description=None` if whoever wrote that server was careless. OpenAI's schema wants a string, so `tool.description or ""` (as above) avoids sending `None` — but also consider: if a discovered tool's description is missing or too vague for the model to tell when to use it, that's a real problem with *that server*, not something your bridge code can silently fix. Worth logging a warning when you hit one, rather than pretending it's fine.

`result.isError` matters, and it's easy to miss. A `CallToolResult` can come back with `.isError = True` — the tool ran, but reported its *own* failure (bad arguments, an internal problem) rather than raising an exception your Python code would catch. If your `call_mcp_tool` only ever looks at `.content`, a failed call can look exactly like a successful one to the model — it'll just see whatever error text the tool happened to put in its content blocks, with no signal that it was actually an error. Check `result.isError` explicitly and label the string differently when it's true (something like `f"Error from tool '{name}': ..."`) — the same "return errors as clear, labeled tool results" instinct Doc06 already taught, just at one more layer of indirection than a plain Python exception.

Sketch both fixes yourself before checking Hint 2.

Difference between Basic, Intermediate, and Advanced: Basic names the two translation directions without any real code. Intermediate gives the real, working conversion in both directions for the case where everything is well-formed. Advanced covers what happens when the *server itself* is imperfect — a missing description, or a tool that reports its own failure through `isError` instead of an exception — which only becomes a real concern once your tools are discovered from someone else's code instead of written by you.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The loop itself, and almost the whole thing

Basic Version

```
messages = [system prompt, the user's task]

repeat up to max_iterations times:
    ask the model to respond, giving it the discovered tools list
    if the model didn't ask for any tool: this is the final answer, stop

    add the model's tool-call request to messages
    for each tool call the model asked for:
        run it through the MCP client (call_mcp_tool)
        add the result to messages as a "tool" message

if the loop finishes without a final answer: that's a MaxIterationsExceeded case
```

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Here's almost the whole loop — the tool-calling round (step 2 of the `for` body) is the part worth writing yourself before checking further:

```
import json
from openai import OpenAI

client = OpenAI()

async def run_agent(task: str, session, openai_tools: list[dict], max_iterations: int):
    messages = [
        {"role": "system", "content": "You are a helpful assistant. Use the available tools to solve the task."},
        {"role": "user", "content": task},
    ]
    log = []

    for step in range(max_iterations):
        response = client.chat.completions.create(
            model="gpt-4o-mini",
            messages=messages,
            tools=openai_tools,
        )
        message = response.choices[0].message

        if not message.tool_calls:
            log.append({"step": step, "type": "final_answer", "content": message.content})
            return {"answer": message.content, "log": log}

        messages.append(message)
        # your turn: for each message.tool_calls entry, parse the JSON
        # arguments, call call_mcp_tool, log the call + its result, and
        # append a {"role": "tool", "tool_call_id": ..., "content": ...}
        # message for each one

    raise RuntimeError(f"Agent did not finish within {max_iterations} steps")
```

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Fill in the tool-calling round, and think about what the *log* should actually capture. A log entry that only says "called a tool" isn't useful later (Step 4's testing, and Doc07's "evaluating an agent" idea) — it should show exactly what arguments were sent and what came back, so a run can be understood after the fact without re-running it:

```
import json
from openai import OpenAI

client = OpenAI()

class MaxIterationsExceeded(Exception):
    def __init__(self, message: str, log: list[dict]):
        super().__init__(message)
        self.log = log

async def run_agent(task: str, session, openai_tools: list[dict], max_iterations:
    messages = [
        {"role": "system", "content": "You are a helpful assistant. Use the avail
        {"role": "user", "content": task},
    ]
    log = []

    for step in range(max_iterations):
        response = client.chat.completions.create(
            model="gpt-4o-mini",
            messages=messages,
            tools=openai_tools,
        )
        message = response.choices[0].message

        if not message.tool_calls:
            log.append({"step": step, "type": "final_answer", "content": message.
            return {"answer": message.content, "log": log}

        messages.append(message)
        for tool_call in message.tool_calls:
            name = tool_call.function.name
            arguments = json.loads(tool_call.function.arguments)
```

```

        log.append({"step": step, "type": "tool_call", "tool": name, "argumen

# your turn: call call_mcp_tool(session, name, arguments),
# log the observation, then append it as a "tool" message
# with tool_call_id=tool_call.id

raise MaxIterationsExceeded(f"Agent did not finish within {max_iterations} st

```

Fill in the marked section, then compare your finished version against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: Basic is the plain shape of the loop, no code. Intermediate is real, runnable code with exactly one gap left for you (the tool-calling round itself — the part that actually reaches into MCP). Advanced adds a named

`MaxIterationsExceeded` exception carrying the partial log, and a log format detailed enough to actually debug a run from later, not just confirm it finished.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)